



Smart Infrastructure Protection and Continuity System

HTTDP Remote Deployment Using Ansible

Author	Ali Yasser Ali Mohamed
Supervisor	Dr. Hisham Zarif

1. Abstract

This report presents an automated deployment workflow using Ansible to remotely configure and manage Linux-based systems from a centralized control server. The demonstrated solution focuses on deploying an Apache HTTP server on a remote CentOS host, however the methodology extends far beyond this single use case. By leveraging Ansible's agentless architecture, idempotent execution, and modular design, this approach provides a scalable and repeatable framework suitable for provisioning virtually any application or infrastructure component across heterogeneous environments. The result is a robust automation pipeline that minimizes manual intervention, reduces configuration errors, accelerates deployment time, and enables consistent system states across multiple servers. This methodology serves as a foundational model that can be extended to large-scale environments, multi-tier architectures, security hardening processes, CI/CD pipelines, and enterprise infrastructure management.

2. Introduction

Modern IT environments demand reliable, efficient, and scalable methods for provisioning and managing server infrastructure. Manual configuration is slow, error-prone, and difficult to repeat consistently across multiple machines. To address these challenges, configuration management tools like Ansible offer a powerful alternative by providing automated, reproducible, and centralized control over system deployments.

This report explores a practical implementation of such automation: deploying and configuring an Apache HTTP server on a remote CentOS virtual machine through an Ubuntu-based Ansible control node. While the example focuses on a single service deployment, the technique itself is far more versatile. Using Ansible's agentless SSH-based control mechanism, YAML-based playbooks, and rich module ecosystem, administrators can automate a wide range of tasks, including:

- System provisioning and initial setup
- Software installation and configuration
- Network and security policy enforcement
- Multi-server orchestration and service coordination
- Application deployment, updates, and version control
- Infrastructure as Code (IaC) for cloud, on-prem, or hybrid environments

3. Environment Overview

- Ubuntu Server (Control Node): IP 10.10.10.11, running Ansible.
- CentOS Server (Managed Node): IP 10.10.10.10, accessible via SSH and with Python installed.

4. Preparing the Ubuntu Control Node

1. Install required packages:

```
sudo apt install software-properties-common sshpass openssh-client -y
```

2. Install the latest Ansible from the official PPA:

```
sudo apt install -t $(lsb_release -sc)-backports ansible
```

5. Preparing the CentOS Managed Node

1. Ensure SSH server is installed and running:

```
dnf install openssh-server -y
```

```
systemctl enable sshd
```

```
systemctl start sshd
```

2. Install Python 3 (required for Ansible):

```
dnf install python3 -y
```

3. Install python3-six to avoid Ansible module errors:

```
dnf install python3-six -y
```

6. Configuring SSH Access

1. Verify manual SSH access from Ubuntu to CentOS:

```
ssh root@10.10.10.10
```

2. Add CentOS host key to known_hosts:

```
mkdir -p ~/.ssh
```

```
ssh-keyscan -H 10.10.10.10 >> ~/.ssh/known_hosts
```

7. Prepare folder structure on Ubuntu

```
sudo mkdir -p /etc/ansible/files
```

```
sudo mkdir -p /etc/ansible/playbooks
```

```
sudo touch /etc/ansible/hosts
```

```
sudo touch /etc/ansible/files/index.html
```

```
sudo touch /etc/ansible/playbooks/deploy_httpd.yml
```

Ansible directory structure will be:

```
/etc/ansible/
```

```
|— hosts
```

```
|— playbooks/
```

```
|   └─ deploy_httpd.yml
```

```
└─ files/
```

```
    └─ index.html
```

8. Creating the Ansible Inventory

Inventory file: /etc/ansible/hosts

```
[centos_clients]
```

```
10.10.10.10 ansible_user=root ansible_password=31415926
```

```
ansible_python_interpreter=/usr/bin/python3
```

9. Testing Ansible Connectivity

```
ansible centos_clients -m ping
```

```
root@pie-virtual-machine:/home/pie# ansible centos_clients -m ping
10.10.10.10 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

10. Deployment Playbook

Playbook path: /etc/ansible/playbooks/deploy-httpd.yml

The playbook installs and configures Apache HTTP server (httpd), deploys custom files, starts the service, and enables it on boot.

- name: Deploy Apache HTTP server and website on CentOS

hosts: centos_clients

become: yes

tasks:

- name: Install Apache (httpd)

yum:

name: httpd

state: present

- name: Ensure Apache is enabled and started

service:

name: httpd

state: started

enabled: yes

- name: Deploy website index.html

copy:

src: /etc/ansible/files/index.html

dest: /var/www/html/index.html

owner: root
group: root
mode: '0644'

- name: Open port 80 in firewalld

firewalld:

port: 80/tcp

permanent: yes

state: enabled

when: ansible_facts['os_family'] == "RedHat"

- name: Reload firewalld

command: firewall-cmd --reload

when: ansible_facts['os_family'] == "RedHat"

11. Executing the Deployment

Run the playbook:

ansible-playbook /etc/ansible/playbooks/deploy-httpd.yml

```
root@pie-virtual-machine:/home/pie# ansible-playbook /etc/ansible/playbooks/deploy_httpd.yml

PLAY [Deploy Apache HTTP server and website on CentOS] *****

TASK [Gathering Facts] *****
ok: [10.10.10.10]

TASK [Install Apache (httpd)] *****
ok: [10.10.10.10]

TASK [Ensure Apache is enabled and started] *****
ok: [10.10.10.10]

TASK [Deploy website index.html] *****
changed: [10.10.10.10]

TASK [Open port 80 in firewallld] *****
changed: [10.10.10.10]

TASK [Reload firewallld] *****
changed: [10.10.10.10]

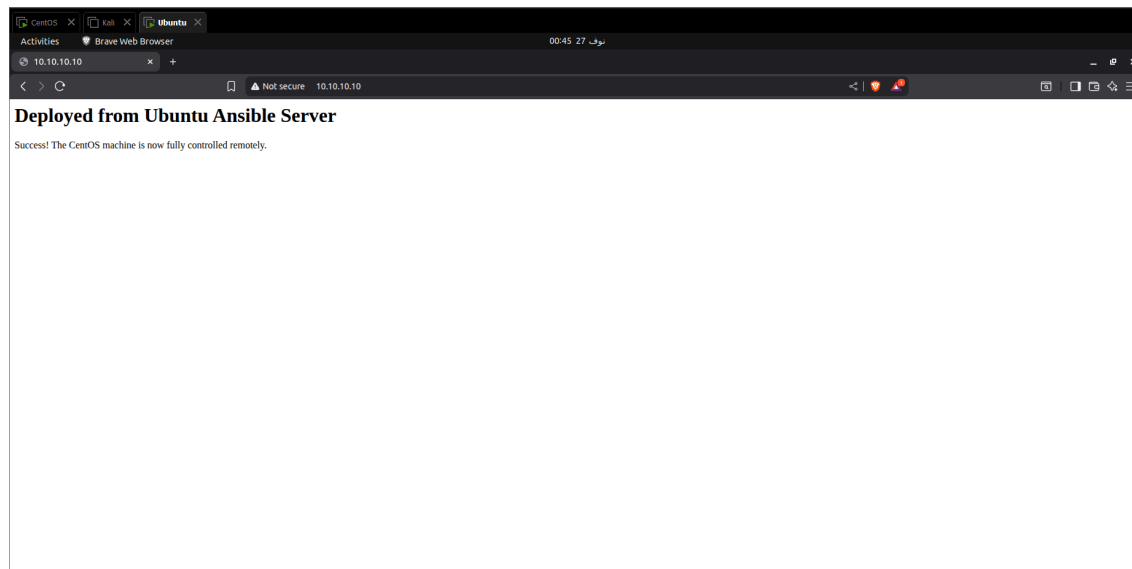
PLAY RECAP *****
10.10.10.10 : ok=6  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

12. Verification

1. Check service status on CentOS:

systemctl status httpd

2. Access the web server from a browser: ***http://10.10.10.10***



13. Conclusion

The deployment was successfully completed. Ansible automated the installation and configuration of Apache HTTP server on the CentOS client, demonstrating effective remote management, infrastructure automation, and configuration management across Linux systems.

14. References

#	Title	Short description	Link
1	Ansible – Playbooks Guide	Introduction to Playbooks	https://docs.ansible.com
2	Ansible – Inventory Guide	Managing Inventories	https://docs.ansible.com
3	Ubuntu – SSH Server Setup	OpenSSH Server	https://ubuntu.com
4	Apache Configuration	Configuring the Server	https://httpd.apache.org
5	YAML Specifications	YAML Specifications	https://yaml.org/spec/
6	OpenSSH Project	SSH Protocol & Manuals	https://www.openssh.com